

Relazione – Gestione dei Permessi di Lettura, Scrittura ed Esecuzione in Linux

Execution Summary

L'obiettivo dell'esercizio è stato configurare e gestire i permessi di lettura (r), scrittura (w) ed esecuzione (x) su un file in ambiente Linux, comprendendo il funzionamento dei comandi principali utilizzati per la gestione dei permessi e analizzando le conseguenze delle modifiche effettuate.

L'attività è stata svolta all'interno di una directory creata appositamente per l'esercizio, in modo da non interferire con file di sistema.

Creazione della directory e del file

Per iniziare, è stata creata una directory denominata `progetto_permessi` utilizzando il comando:

`mkdir progetto_permessi`

Il comando `mkdir` (make directory) permette di creare una nuova directory nel percorso corrente.

Successivamente, ci si è spostati all'interno della directory con:

`cd progetto_permessi`

Il comando `cd` (change directory) consente di cambiare la directory di lavoro.

Infine, è stato creato un file vuoto denominato **`relazione.txt`** tramite:

`touch relazione.txt`

Il comando `touch` crea un nuovo file vuoto se non esiste oppure aggiorna la data di modifica se il file è già presente.

Verifica dei permessi iniziali

Per verificare i permessi assegnati automaticamente dal sistema al momento della creazione del file, è stato utilizzato il comando:

ls -l

Il comando **ls** elenca i file presenti nella directory, mentre l'opzione **-l** (long listing) mostra informazioni dettagliate, tra cui:

- tipo di file
- permessi
- numero di collegamenti
- proprietario
- gruppo
- dimensione
- data di ultima modifica

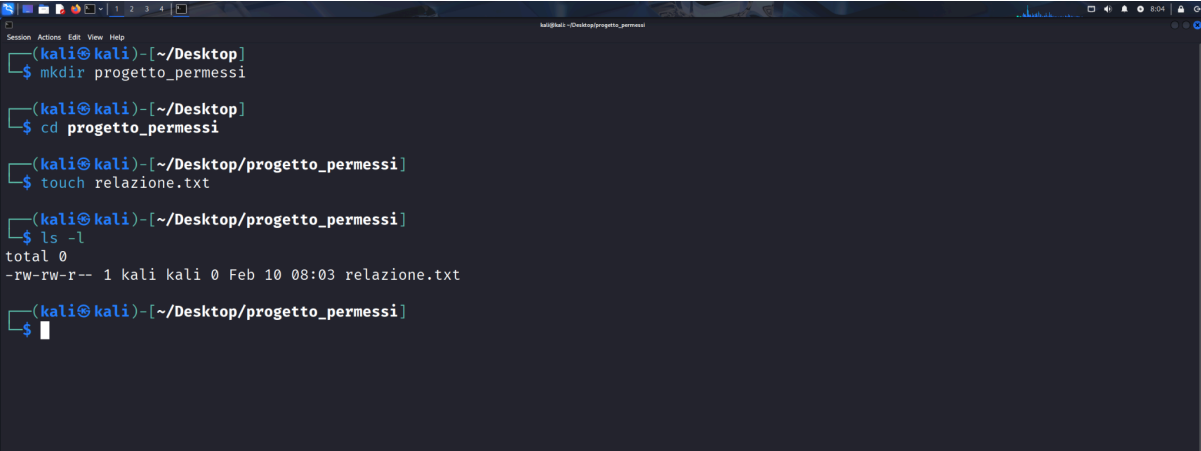
L'output iniziale risultava:

-rw-rw-r-- 1 kali kali 0 Feb 10 08:03 relazione.txt

Analizzando i permessi:

- rw- : il proprietario (kali) può leggere e scrivere
- rw- : il gruppo può leggere e scrivere
- r-- : gli altri utenti possono solo leggere

Figura 1



```
Session: Actions Edit View Help
kali@kali: ~/Desktop/progetto_permessi
(kali@kali)~-[~/Desktop]
$ mkdir progetto_permessi
(kali@kali)~-[~/Desktop]
$ cd progetto_permessi
(kali@kali)~-[~/Desktop/progetto_permessi]
$ touch relazione.txt
(kali@kali)~-[~/Desktop/progetto_permessi]
$ ls -l
total 0
-rw-rw-r-- 1 kali kali 0 Feb 10 08:03 relazione.txt
(kali@kali)~-[~/Desktop/progetto_permessi]
$
```

Analisi della situazione iniziale

Inizialmente il file permetteva la scrittura anche al gruppo.

Questo significa che tutti gli utenti appartenenti allo stesso gruppo del proprietario potevano modificare il contenuto del file.

Inoltre, gli altri utenti del sistema potevano leggere il file, anche se non modificarlo.

Dal punto di vista della sicurezza, questa configurazione non garantisce un controllo completo sull'accesso al contenuto.

Modifica dei permessi

L'obiettivo era limitare i permessi nel seguente modo:

- u :user (proprietario)
- g :group (gruppo)
- o :others (altri utenti)
- + :aggiunge permessi
- - :rimuove permessi
- = :assegna esattamente i permessi indicati

Sono stati eseguiti i seguenti comandi:

chmod g-w relazione.txt

chmod o-r relazione.txt

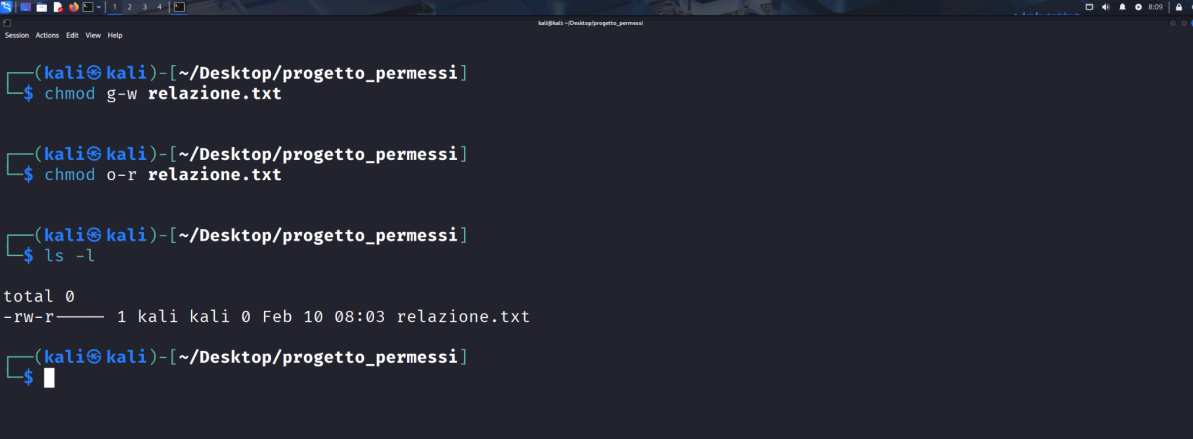
Spiegazione:

- chmod g-w rimuove il permesso di scrittura al gruppo
- chmod o-r rimuove il permesso di lettura agli altri utenti

Dopo la modifica, verificando nuovamente con **ls -l**, l'output risultava:

-rw-r----- 1 kali kali 0 Feb 10 08:03 relazione.txt

Figura 2.



```
(kali㉿kali)-[~/Desktop/progetto_permessi]
$ chmod g-w relazione.txt

(kali㉿kali)-[~/Desktop/progetto_permessi]
$ chmod o-r relazione.txt

(kali㉿kali)-[~/Desktop/progetto_permessi]
$ ls -l

total 0
-rw-r----- 1 kali kali 0 Feb 10 08:03 relazione.txt

(kali㉿kali)-[~/Desktop/progetto_permessi]
$
```

Motivazione delle scelte effettuate

Le modifiche ai permessi sono state effettuate per motivi di sicurezza e controllo degli accessi.

In particolare:

- È stata limitata la modifica del file al solo proprietario, evitando che utenti appartenenti allo stesso gruppo potessero alterarne il contenuto.
- È stato rimosso l'accesso agli altri utenti per evitare lettura non autorizzata del file.

Questa configurazione garantisce che solo il proprietario abbia pieno controllo del file, mentre il gruppo può consultarlo senza modificarlo, e gli altri utenti non possono accedervi

Test dei permessi

Per verificare il corretto funzionamento delle restrizioni impostate, sono stati effettuati test di accesso.

Ad esempio, tentando di eseguire il file:

`./relazione.txt`

Il sistema ha restituito:

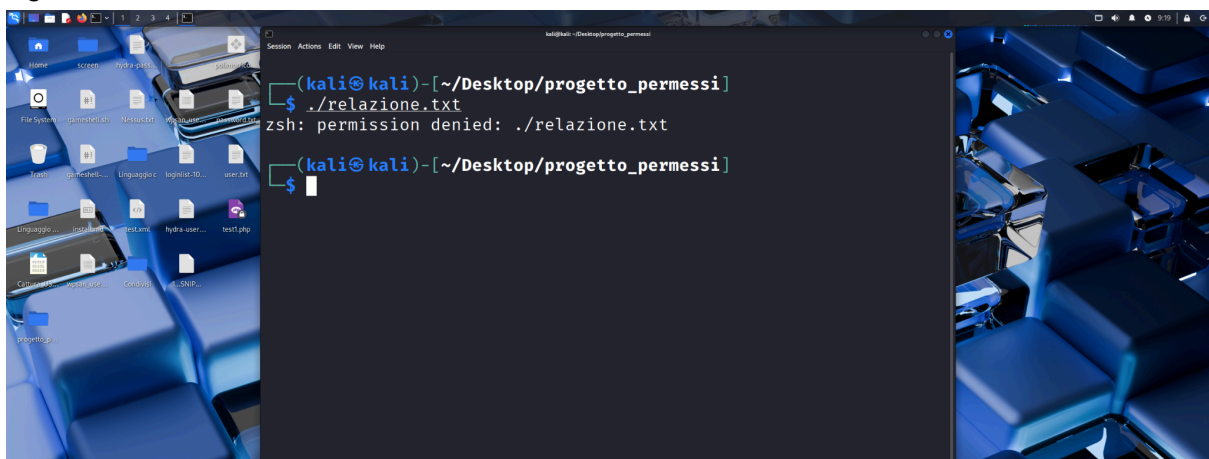
Permission denied

Questo conferma che il file non possiede il permesso di esecuzione.

I test hanno dimostrato che:

- Il proprietario mantiene i diritti di lettura e scrittura.
- Il gruppo può solo leggere.
- Gli altri utenti non possono accedere al file.

Figura 3.



Approfondimento Teorico

Il Modello dei Permessi nei Sistemi Unix/Linux

Nei sistemi Unix e Linux, ogni file e directory possiede un insieme di permessi che regolano l'accesso secondo tre categorie di utenti:

- **User (u)** :proprietario del file
- **Group (g)** :gruppo associato al file
- **Others (o)** :tutti gli altri utenti del sistema

Per ciascuna categoria possono essere assegnati tre tipi di permesso:

- **r (read)** :consente la lettura del contenuto
- **w (write)** :consente la modifica del contenuto
- **x (execute)** :consente l'esecuzione (per file) o l'accesso (per directory)

Questo modello rappresenta uno dei pilastri della sicurezza nei sistemi Unix-like, in quanto permette una gestione granulare e strutturata degli accessi.

Modalità Numerica vs Modalità Simbolica

Il comando **chmod** può essere utilizzato in due modalità principali:

1.Modalità Numerica (ottale)

In questa modalità, i permessi vengono rappresentati con valori numerici:

- $r = 4$
- $w = 2$
- $x = 1$

La somma dei valori determina il permesso assegnato:

- $7 = rwx$
- $6 = rw-$
- $5 = r-x$
- $4 = r--$

Ad esempio:

chmod 640 relazione.txt

assegna:

- 6 (rw-) al proprietario
- 4 (r--) al gruppo
- 0 (---) agli altri

La modalità numerica è utile quando si vuole impostare rapidamente l'intera configurazione dei permessi in un solo comando.

2. Modalità Simbolica

Nella modalità simbolica si interviene in modo più mirato utilizzando:

- u (user)
- g (group)
- o (others)
- + (aggiunge)
- - (rimuove)
- = (assegna esattamente)

Conclusione

L'esercizio ha permesso di comprendere in modo pratico il funzionamento dei permessi in ambiente Linux e l'importanza del comando **chmod** per la gestione della sicurezza.

Attraverso l'analisi dei permessi iniziali e la loro modifica mirata, è stato possibile migliorare il livello di protezione del file, limitando l'accesso in modo coerente con il principio del minimo privilegio.

La gestione corretta dei permessi rappresenta un elemento fondamentale nell'amministrazione di sistemi Linux, in quanto consente di controllare in modo preciso chi può leggere, modificare o eseguire file e directory.